

Lab 1

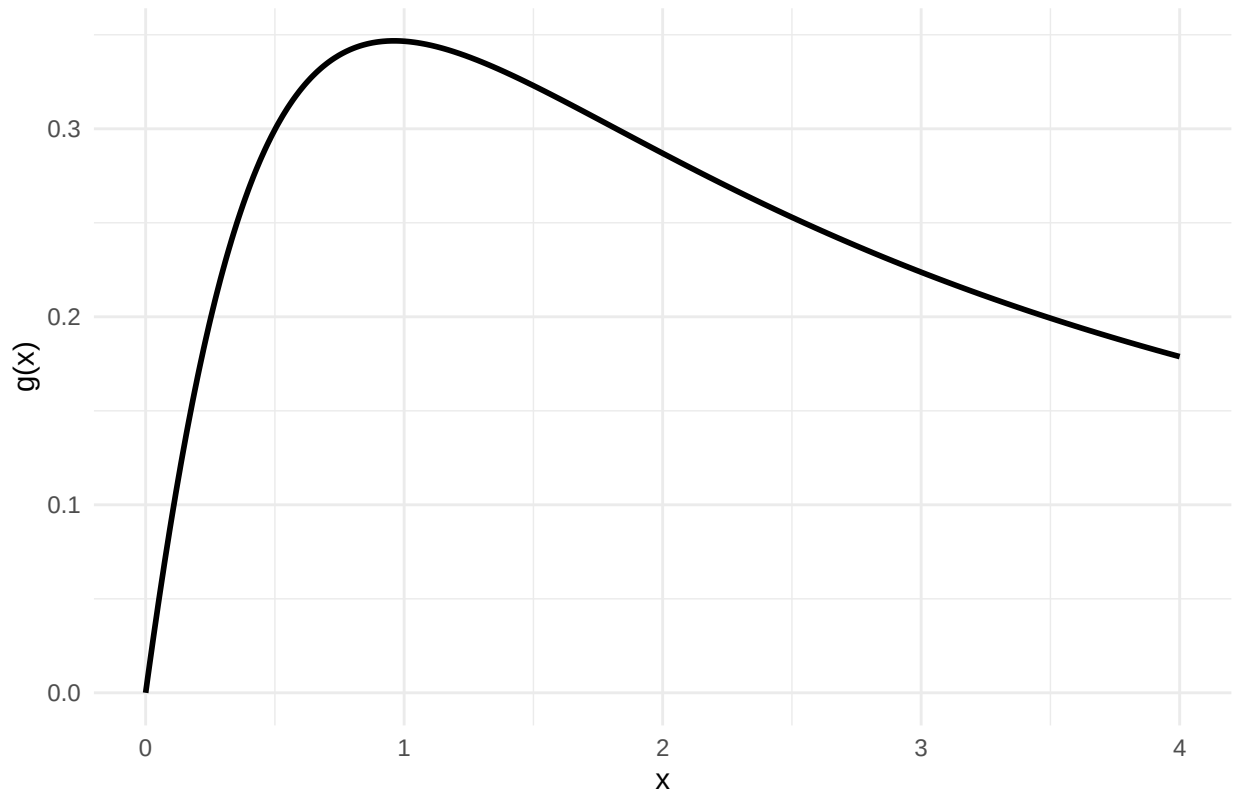
Aron Enström (aroen488), Sergey Volchkov (servo519)

2026-01-22

1

1.1. Function plot

$$g(x) = \log(x+1)/(x^{3/2}+1)$$



The function maximum should be at around 1.

1.2. Derivative and plot

$$g(x) = \frac{\log(x+1)}{x^{3/2}+1}.$$

We set $u(x) = \log(x+1)$ and $v(x) = x^{3/2} + 1$, so $g(x) = \frac{u(x)}{v(x)}$.

By the quotient rule,

$$g'(x) = \frac{u'(x)v(x) - u(x)v'(x)}{(v(x))^2}.$$

Then $u'(x) = \frac{1}{x+1}$ and $v'(x) = \frac{3}{2}x^{1/2}$.

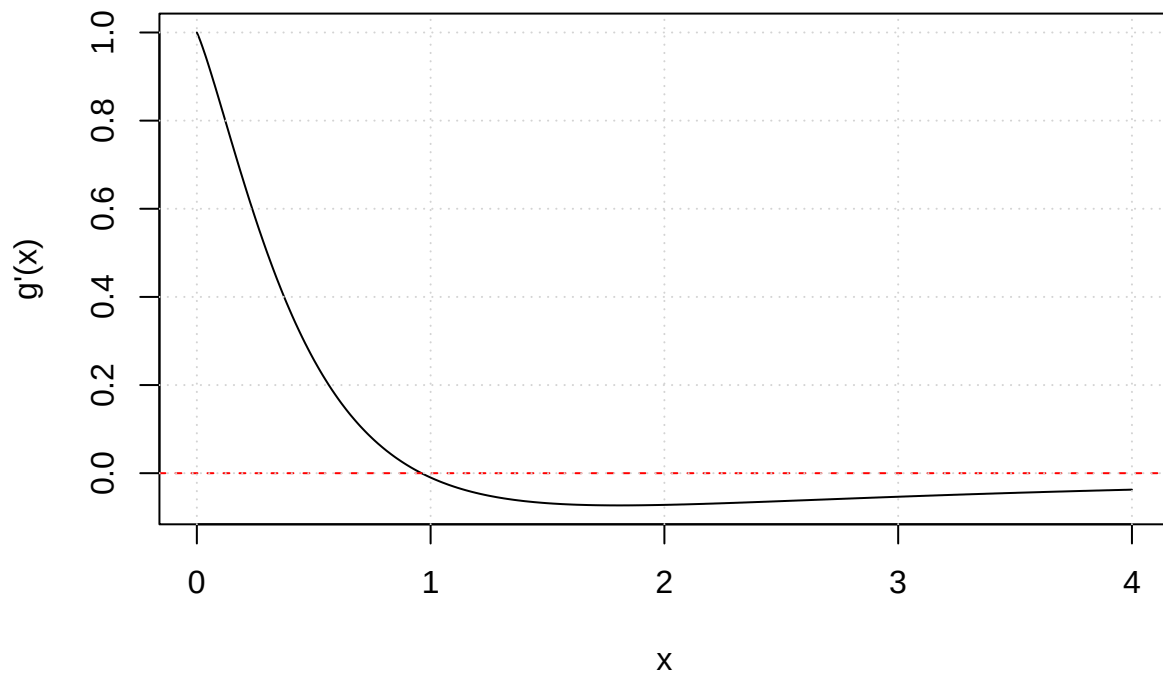
Which gives:

$$g'(x) = \frac{\frac{1}{x+1}(x^{3/2} + 1) - \log(x+1) \left(\frac{3}{2}x^{1/2}\right)}{(x^{3/2} + 1)^2}.$$

Or equivalently,

$$g'(x) = \frac{(x^{3/2} + 1) - \frac{3}{2}x^{1/2}(x+1)\log(x+1)}{(x+1)(x^{3/2} + 1)^2}.$$

Derivative g'(x) on [0, 4]



1.3. Bisection method implementation

Test results for $g(x)$ using the bisection method on the interval $[0, 4]$:

```
## $x_star
## [1] 0.9610603
##
## $g_at_x_star
## [1] 0.3467708
##
## $type
## [1] "local maximum (g' changes + to -)"
##
## $details
## $details$root
## [1] 0.9610603
```

```
##
## $details$f_root
## [1] -2.265823e-11
##
## $details$iter
## [1] 32
##
## $details$a
## [1] 0.9610603
##
## $details$b
## [1] 0.9610603
```

1.4. Secant method implementation

Test results for $g(x)$ using the secant method on the interval $[0.2, 1]$:

```
## $x_star
## [1] 0.9610603
##
## $g_at_x_star
## [1] 0.3467708
##
## $type
## [1] "local maximum (g' changes + to -)"
##
## $details
## $details$root
## [1] 0.9610603
##
## $details$f_root
## [1] 5.138024e-12
##
## $details$iter
## [1] 5
##
## $details$x0
## [1] 0.9610605
##
## $details$x1
## [1] 0.9610603
```

1.5. Comparison of the two methods for the task at hand

##	method	start1	start2	status	root	true_root_found	iterations
## 1	bisection	0.0	0.5	fail	NA	FALSE	NA
## 2	bisection	0.0	1.0	ok	9.610603e-01	TRUE	30
## 4	bisection	0.0	1.5	ok	9.610603e-01	TRUE	29
## 7	bisection	0.0	2.0	ok	9.610603e-01	TRUE	31
## 11	bisection	0.0	2.5	ok	9.610603e-01	TRUE	32
## 16	bisection	0.0	3.0	ok	9.610603e-01	TRUE	30
## 22	bisection	0.0	3.5	ok	9.610603e-01	TRUE	29
## 29	bisection	0.0	4.0	ok	9.610603e-01	TRUE	32
## 3	bisection	0.5	1.0	ok	9.610603e-01	TRUE	29
## 5	bisection	0.5	1.5	ok	9.610603e-01	TRUE	30

## 8	bisection	0.5	2.0	ok	9.610603e-01	TRUE	31
## 12	bisection	0.5	2.5	ok	9.610603e-01	TRUE	31
## 17	bisection	0.5	3.0	ok	9.610603e-01	TRUE	31
## 23	bisection	0.5	3.5	ok	9.610603e-01	TRUE	32
## 30	bisection	0.5	4.0	ok	9.610603e-01	TRUE	32
## 6	bisection	1.0	1.5	fail	NA	FALSE	NA
## 9	bisection	1.0	2.0	fail	NA	FALSE	NA
## 13	bisection	1.0	2.5	fail	NA	FALSE	NA
## 18	bisection	1.0	3.0	fail	NA	FALSE	NA
## 24	bisection	1.0	3.5	fail	NA	FALSE	NA
## 31	bisection	1.0	4.0	fail	NA	FALSE	NA
## 10	bisection	1.5	2.0	fail	NA	FALSE	NA
## 14	bisection	1.5	2.5	fail	NA	FALSE	NA
## 19	bisection	1.5	3.0	fail	NA	FALSE	NA
## 25	bisection	1.5	3.5	fail	NA	FALSE	NA
## 32	bisection	1.5	4.0	fail	NA	FALSE	NA
## 15	bisection	2.0	2.5	fail	NA	FALSE	NA
## 20	bisection	2.0	3.0	fail	NA	FALSE	NA
## 26	bisection	2.0	3.5	fail	NA	FALSE	NA
## 33	bisection	2.0	4.0	fail	NA	FALSE	NA
## 21	bisection	2.5	3.0	fail	NA	FALSE	NA
## 27	bisection	2.5	3.5	fail	NA	FALSE	NA
## 34	bisection	2.5	4.0	fail	NA	FALSE	NA
## 28	bisection	3.0	3.5	fail	NA	FALSE	NA
## 35	bisection	3.0	4.0	fail	NA	FALSE	NA
## 36	bisection	3.5	4.0	fail	NA	FALSE	NA
## 37	secant	0.0	0.5	ok	9.610603e-01	TRUE	8
## 38	secant	0.0	1.0	ok	9.610603e-01	TRUE	5
## 40	secant	0.0	1.5	ok	9.610603e-01	TRUE	12
## 43	secant	0.0	2.0	ok	3.213281e+04	FALSE	34
## 47	secant	0.0	2.5	ok	3.366863e+04	FALSE	36
## 52	secant	0.0	3.0	ok	3.407846e+04	FALSE	36
## 58	secant	0.0	3.5	ok	3.601270e+04	FALSE	36
## 65	secant	0.0	4.0	ok	3.018935e+04	FALSE	35
## 39	secant	0.5	1.0	ok	9.610603e-01	TRUE	5
## 41	secant	0.5	1.5	ok	9.610603e-01	TRUE	10
## 44	secant	0.5	2.0	ok	3.417835e+04	FALSE	28
## 48	secant	0.5	2.5	ok	3.443344e+04	FALSE	36
## 53	secant	0.5	3.0	ok	3.366477e+04	FALSE	36
## 59	secant	0.5	3.5	ok	3.524994e+04	FALSE	36
## 66	secant	0.5	4.0	ok	2.947128e+04	FALSE	35
## 42	secant	1.0	1.5	ok	9.610603e-01	TRUE	7
## 45	secant	1.0	2.0	ok	9.610603e-01	TRUE	8
## 49	secant	1.0	2.5	fail	NA	FALSE	NA
## 54	secant	1.0	3.0	ok	3.456489e+04	FALSE	38
## 60	secant	1.0	3.5	ok	3.470520e+04	FALSE	38
## 67	secant	1.0	4.0	fail	NA	FALSE	NA
## 46	secant	1.5	2.0	fail	NA	FALSE	NA
## 50	secant	1.5	2.5	ok	3.321995e+04	FALSE	32
## 55	secant	1.5	3.0	ok	3.378095e+04	FALSE	34
## 61	secant	1.5	3.5	ok	3.116072e+04	FALSE	34
## 68	secant	1.5	4.0	ok	3.097144e+04	FALSE	34
## 51	secant	2.0	2.5	ok	3.531694e+04	FALSE	35
## 56	secant	2.0	3.0	ok	3.468978e+04	FALSE	35

## 62	secant	2.0	3.5	ok 3.543345e+04	FALSE	35
## 69	secant	2.0	4.0	ok 3.672040e+04	FALSE	35
## 57	secant	2.5	3.0	ok 3.403296e+04	FALSE	35
## 63	secant	2.5	3.5	ok 3.519499e+04	FALSE	35
## 70	secant	2.5	4.0	ok 3.668602e+04	FALSE	35
## 64	secant	3.0	3.5	ok 3.567116e+04	FALSE	35
## 71	secant	3.0	4.0	ok 2.921852e+04	FALSE	34
## 72	secant	3.5	4.0	ok 2.982723e+04	FALSE	34

The bisection method fails when the starting interval does not include the true root.

The secant method can succeed even when the starting interval does not include the true root and takes fewer iterations (up to 12 vs around 30 for the bisection method).

Using the update formula for the secant method

$$x_{n+1} = x_n - f(x_n) \frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}.$$

which in our case is equivalent to

$$x_{n+1} = x_n - g'(x_n) \frac{x_n - x_{n-1}}{g'(x_n) - g'(x_{n-1})}.$$

The pattern in the table for the secant method (lots of roots around 10^4 or “fail”) is consistent with secant taking a bad step and then wandering to a region where $g'(x)$ is very small/flat (we can observe it in the region of the $g'(x)$ graph in section 1.2 for in the x interval from 1.5 to 4 and we can see from the table that the algorithm fails in nearly all cases when one or both starting values are in this interval) or where the update becomes ill-conditioned (e.g. $g'(x_n) - g'(x_{n-1})$ tiny), which can blow up the update and lead to divergence or stagnation. This is a known secant pitfall: it can diverge when it encounters a near-zero slope / derivative-like quantity in the update.

Algorithm converged: TRUE

Reason: tolerance reached

Number of iterations: 42

History - first 10 iterations:

##	iter	x0	x1	f_x0	f_x1	denom	x2
## 1	1	0.000000	2.000000	1.0000000000	-0.0719367338	-1.0719367338	1.865782
## 2	2	2.000000	1.865782	-0.0719367338	-0.0729762436	-0.0010395098	11.288245
## 3	3	1.865782	11.288245	-0.0729762436	-0.0062531156	0.0667231280	12.171294
## 4	4	11.288245	12.171294	-0.0062531156	-0.0053951572	0.0008579584	17.724226
## 5	5	12.171294	17.724226	-0.0053951572	-0.0025293110	0.0028658462	22.625080
## 6	6	17.724226	22.625080	-0.0025293110	-0.0015227392	0.0010065719	30.039079
## 7	7	22.625080	30.039079	-0.0015227392	-0.0008348680	0.0006878711	39.037437
## 8	8	30.039079	39.037437	-0.0008348680	-0.0004745677	0.0003603004	50.889574
## 9	9	39.037437	50.889574	-0.0004745677	-0.0002659422	0.0002086254	65.997914
## 10	10	50.889574	65.997914	-0.0002659422	-0.0001497870	0.0001161552	85.480754
##		f_x2					
## 1		-7.297624e-02					
## 2		-6.253116e-03					
## 3		-5.395157e-03					
## 4		-2.529311e-03					
## 5		-1.522739e-03					
## 6		-8.348680e-04					

```
## 7 -4.745677e-04
## 8 -2.659422e-04
## 9 -1.497870e-04
## 10 -8.416287e-05
```

History - last 10 iterations:

```
##      iter      x0      x1      f_x0      f_x1      denom      x2
## 33   33  15533.31  19797.04 -4.481354e-10 -2.509786e-10  1.971568e-10  25224.73
## 34   34  19797.04  25224.73 -2.509786e-10 -1.405490e-10  1.104296e-10  32132.81
## 35   35  25224.73  32132.81 -1.405490e-10 -7.870170e-11  6.184731e-11  40923.45
## 36   36  32132.81  40923.45 -7.870170e-11 -4.406637e-11  3.463533e-11  52107.74
## 37   37  40923.45  52107.74 -4.406637e-11 -2.467168e-11  1.939469e-11  66335.10
## 38   38  52107.74  66335.10 -2.467168e-11 -1.381211e-11  1.085958e-11  84430.63
## 39   39  66335.10  84430.63 -1.381211e-11 -7.732002e-12  6.080103e-12  107442.52
## 40   40  84430.63  107442.52 -7.732002e-12 -4.328089e-12  3.403913e-12  136702.24
## 41   41  107442.52  136702.24 -4.328089e-12 -2.422556e-12  1.905534e-12  173900.90
## 42   42  136702.24  173900.90 -2.422556e-12 -1.355894e-12  1.066662e-12  221186.20
##      f_x2
## 33 -1.405490e-10
## 34 -7.870170e-11
## 35 -4.406637e-11
## 36 -2.467168e-11
## 37 -1.381211e-11
## 38 -7.732002e-12
## 39 -4.328089e-12
## 40 -2.422556e-12
## 41 -1.355894e-12
## 42 -7.588447e-13
```

We can see from the above output that with the starting values 0 and 2, due to the above-mentioned flatness of the $g'(x)$ graph, the secant algorithm overshoots the interval containing the root and then wanders further and further away from the interval containing the root as the numerator $x_n - x_{n-1}$ of the above equation becomes larger and larger while the denominator $g'(x_n) - g'(x_{n-1})$ becomes smaller and smaller.

The amount of programming effort is perhaps slightly higher for the secant method.

1.6. General comparison of the bisection and secant methods

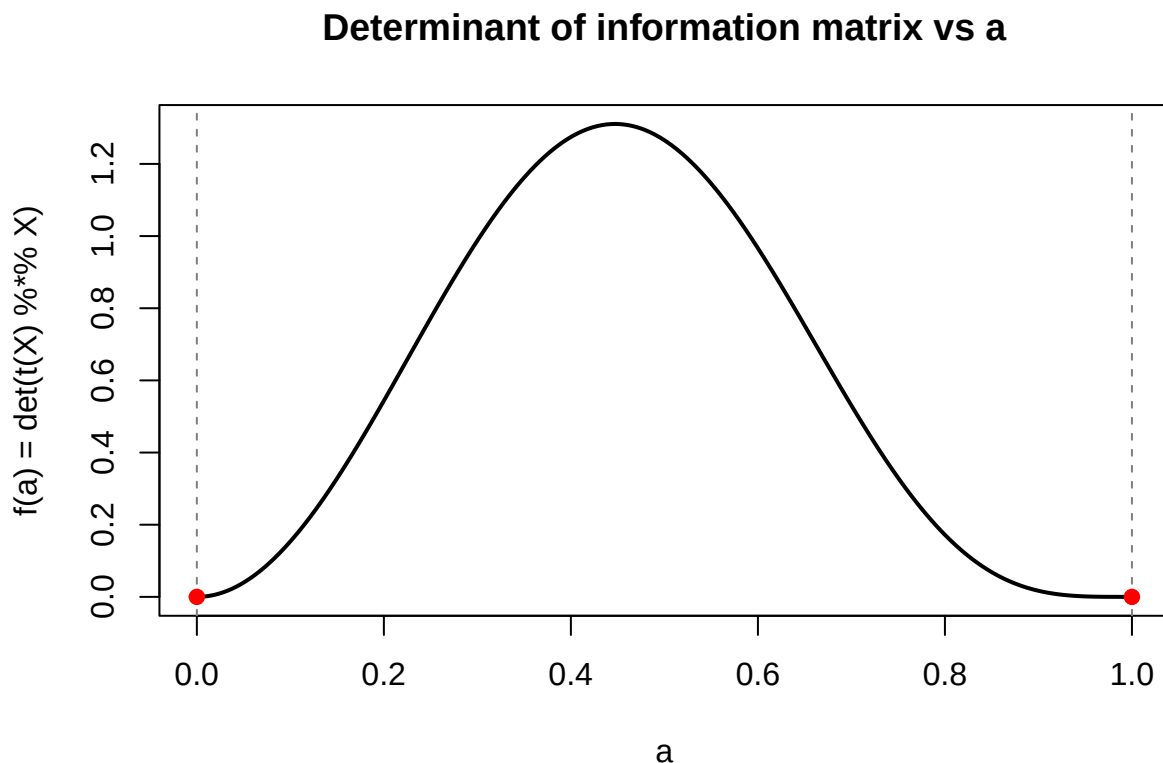
For the task at hand, we would choose the bisection method despite it being slower than the secant method, we can determine the region containing the true root quite reliably, while the secant method appears to be rather sensitive to the choice of the starting values due to the flatness of the derivative function.

Generally, we prefer bisection when we have a bracketing interval $[a, b]$ with a sign change $f(a)f(b) < 0$, and we want a method that is guaranteed to converge under mild conditions (continuous f). We prefer secant when we have good starting values (close to the root), function evaluations are costly and we want fewer iterations than bisection.

2

2.1. Function to calculate determinant of the information matrix

2.2. Function plot



Interpretation of $a=0$ and $a=1$

- Case $a = 0$: the design points are $x = (-1, 0, 0, 1)$, so $x = 0$ is duplicated and there are only three distinct x -values. This typically makes the design matrix X lose rank, so the information matrix $X^\top X$ becomes singular and $f(0) = \det(X^\top X) = 0$, meaning the cubic regression parameters are not all identifiable from this design.
- Case $a = 1$: the design points are $x = (-1, -1, 1, 1)$, so there are only two distinct x -values (each duplicated). Then X cannot have full column rank 4, so $X^\top X$ is singular and $f(1) = \det(X^\top X) = 0$.

2.3. Local maximum of the function

The function has its maximum at 0.4472098 with the maximum value of 1.31072. This matches what we see on the plot in 2.2.

Appendix

```
#1.1
library(ggplot2)
g <- function(x) log(x + 1) / (x^(3/2) + 1)

df <- data.frame(x = seq(0, 4, length.out = 1000))
```

```

df$g <- g(df$x)

ggplot(df, aes(x, g)) +
  geom_line(linewidth = 1) +
  labs(
    title = "g(x) = log(x+1)/(x^(3/2)+1)",
    x = "x", y = "g(x)"
  ) +
  theme_minimal()
#1.2
g_prime <- function(x) {
  ((x^(3/2) + 1)/(x + 1) - (3/2) * x^(1/2) * log(x + 1)) / (x^(3/2) + 1)^2
}

curve(g_prime(x), from = 0, to = 4, n = 1000,
      xlab = "x", ylab = "g'(x)",
      main = "Derivative g'(x) on [0, 4]")

abline(h = 0, lty = 2, col = "red")
grid()
#1.3

## Bisection method (root-finding) for interval [a, b]
bisect_root <- function(f, a, b, tol = 1e-10, max_iter = 200) {
  fa <- f(a); fb <- f(b)

  if (!is.finite(fa) || !is.finite(fb))
    stop("f(a) or f(b) is not finite.")

  if (fa == 0) return(list(root = a, f_root = fa, iter = 0, a = a, b = b))
  if (fb == 0) return(list(root = b, f_root = fb, iter = 0, a = a, b = b))

  ## Need a sign change for bisection
  if (fa * fb > 0)
    stop("No sign change on [a, b]. Choose an interval where f(a) and f(b) have opposite signs.")

  for (iter in 1:max_iter) {
    c <- (a + b) / 2
    fc <- f(c)

    if (!is.finite(fc))
      stop("f(midpoint) not finite; try a different interval.")

    if (abs(fc) < tol || (b - a) / 2 < tol)
      return(list(root = c, f_root = fc, iter = iter, a = a, b = b))

    if (fa * fc < 0) {
      b <- c; fb <- fc
    } else {
      a <- c; fa <- fc
    }
  }
}

```



```

    list(root = (a + b)/2, f_root = f((a + b)/2), iter = max_iter, a = a, b = b)
}

## Find a critical point of g by solving g'(x)=0 on a user-selected interval
find_local_max_g <- function(a, b, tol = 1e-10, max_iter = 200) {
  out <- bisection_root(g_prime, a, b, tol = tol, max_iter = max_iter)
  x_star <- out$root

  ## First-derivative sign change test: + to - indicates local max
  eps <- min(1e-4, (b - a) / 100)
  left <- g_prime(max(a, x_star - eps))
  right <- g_prime(min(b, x_star + eps))

  type <- if (is.finite(left) && is.finite(right) && left > 0 && right < 0) {
    "local maximum (g' changes + to -)"
  } else if (is.finite(left) && is.finite(right) && left < 0 && right > 0) {
    "local minimum (g' changes - to +)"
  } else {
    "critical point found; sign-change unclear on chosen interval"
  }

  list(x_star = x_star, g_at_x_star = g(x_star), type = type, details = out)
}

## Test
a = 0
b = 4
res <- find_local_max_g(a = a, b = b)
res
#1.4
## Secant method for root finding
secant_root <- function(f, x0, x1, tol = 1e-10, max_iter = 200) {
  f0 <- f(x0); f1 <- f(x1)

  if (!is.finite(f0) || !is.finite(f1))
    stop("f(x0) or f(x1) is not finite.")

  if (f0 == 0) return(list(root = x0, f_root = f0, iter = 0, x0 = x0, x1 = x1))
  if (f1 == 0) return(list(root = x1, f_root = f1, iter = 0, x0 = x0, x1 = x1))

  for (iter in 1:max_iter) {
    denom <- (f1 - f0)
    if (denom == 0) stop("Secant breakdown: f(x1) - f(x0) = 0.")

    ## x_{n+1} = x_n - f(x_n) (x_n - x_{n-1}) / (f(x_n) - f(x_{n-1}))
    x2 <- x1 - f1 * (x1 - x0) / denom

    if (!is.finite(x2)) stop("Non-finite iterate produced; try different starting values.")

    if (abs(x2 - x1) < tol)
      return(list(root = x2, f_root = f(x2), iter = iter, x0 = x0, x1 = x1))

    ## shift (x0,f0) <- (x1,f1), (x1,f1) <- (x2,f2)
  }
}

```

```

x0 <- x1; f0 <- f1
x1 <- x2; f1 <- f(x1)

if (!is.finite(f1)) stop("Non-finite f(x) encountered; try different starting values.")
if (abs(f1) < tol) # optional early stop on function value
  return(list(root = x1, f_root = f1, iter = iter, x0 = x0, x1 = x1))
}

list(root = x1, f_root = f1, iter = max_iter, x0 = x0, x1 = x1)
}

## Find local max of g by applying secant to g'
find_local_max_g_secant <- function(x0, x1, tol = 1e-10, max_iter = 200) {
  out <- secant_root(g_prime, x0, x1, tol = tol, max_iter = max_iter)
  x_star <- out$root

  ## First-derivative sign-change check (heuristic)
  eps <- min(1e-4, abs(x1 - x0) / 100)
  left <- g_prime(x_star - eps)
  right <- g_prime(x_star + eps)

  type <- if (is.finite(left) && is.finite(right) && left > 0 && right < 0) {
    "local maximum (g' changes + to -)"
  } else if (is.finite(left) && is.finite(right) && left < 0 && right > 0) {
    "local minimum (g' changes - to +)"
  } else {
    "critical point found; sign-change unclear from starting values"
  }

  list(x_star = x_star, g_at_x_star = g(x_star), type = type, details = out)
}

## Test
x0 = 0.2
x1 = 1
res <- find_local_max_g_secant(x0 = x0, x1 = x1)
res
## 1.5
# The "true" maximizer x* on [0,4]
# Find sign changes of g' on a fine grid, then bisection in the first bracket.
grid <- seq(0, 4, length.out = 2001)
vals <- g_prime(grid)

idx <- which(diff(sign(vals)) != 0) # sign changes
if (length(idx) == 0) stop("No sign change found on [0,4]; cannot bracket a root.")

a_star <- grid[idx[1]]
b_star <- grid[idx[1] + 1]
true_out <- bisection_root(g_prime, a_star, b_star, tol = 1e-14, max_iter = 2000)
x_true <- true_out$root

## Bisection: test many [a,b]
bisection_grid <- expand.grid(

```

```

a = seq(0, 3.5, by = 0.5),
b = seq(0.5, 4.0, by = 0.5)
)
bisect_grid <- subset(bisect_grid, a < b)

## Secant: test many (x0,x1) pairs
secant_grid <- expand.grid(
  x0 = seq(0.0, 3.5, by = 0.5),
  x1 = seq(0.5, 4.0, by = 0.5)
)
secant_grid <- subset(secant_grid, x0 < x1)

## Helper: run safely and return a row
run_one <- function(method, p1, p2, tol_root = 1e-4) {
  if (method == "bisection") {
    out <- tryCatch(bisect_root(g_prime, p1, p2), error = function(e) NULL)
    root <- if (is.null(out)) NA_real_ else out$root
    iter <- if (is.null(out)) NA_integer_ else out$iter
    status <- if (is.null(out)) "fail" else "ok"
    data.frame(method = method, start1 = p1, start2 = p2,
               status = status,
               root = root,
               true_root_found = isTRUE(!is.na(root) && abs(root - x_true) <= tol_root),
               iterations = iter)
  } else {
    out <- tryCatch(secant_root(g_prime, p1, p2), error = function(e) NULL)
    root <- if (is.null(out)) NA_real_ else out$root
    iter <- if (is.null(out)) NA_integer_ else out$iter
    status <- if (is.null(out)) "fail" else "ok"
    data.frame(method = method, start1 = p1, start2 = p2,
               status = status,
               root = root,
               true_root_found = isTRUE(!is.na(root) && abs(root - x_true) <= tol_root),
               iterations = iter)
  }
}

## Run and bind results
bisect_res <- do.call(rbind, lapply(seq_len(nrow(bisect_grid)), function(i)
  run_one("bisection", bisect_grid$a[i], bisect_grid$b[i])
))

secant_res <- do.call(rbind, lapply(seq_len(nrow(secant_grid)), function(i)
  run_one("secant", secant_grid$x0[i], secant_grid$x1[i])
))

results <- rbind(bisect_res, secant_res)

## Sort for readability
results <- results[order(results$method, results$start1, results$start2), ]

## Print summary table
results

```

```

# Investigate the cause of failure of the secant method
secant_trace <- function(f, x0, x1, tol = 1e-12, max_iter = 50) {
  hist <- vector("list", max_iter)

  f0 <- f(x0); f1 <- f(x1)
  if (!is.finite(f0) || !is.finite(f1)) stop("Non-finite start.")

  for (iter in 1:max_iter) {
    denom <- f1 - f0

    x2 <- if (denom == 0) NA_real_ else x1 - f1 * (x1 - x0) / denom

    hist[[iter]] <- data.frame(
      iter = iter,
      x0 = x0, x1 = x1,
      f_x0 = f0, f_x1 = f1,
      denom = denom,
      x2 = x2,
      f_x2 = if (is.finite(x2)) f(x2) else NA_real_
    )

    if (!is.finite(x2))
      return(list(converged = FALSE, reason = "non-finite x2 (likely denom ~ 0 or overflow)",
                  iter = iter, history = do.call(rbind, hist[1:iter])))

    if (abs(x2 - x1) < tol || abs(f(x2)) < tol)
      return(list(converged = TRUE, reason = "tolerance reached",
                  iter = iter, root = x2, f_root = f(x2),
                  history = do.call(rbind, hist[1:iter])))

    ## shift
    x0 <- x1; f0 <- f1
    x1 <- x2; f1 <- f(x1)
    if (!is.finite(f1))
      return(list(converged = FALSE, reason = "non-finite f(x)",
                  iter = iter, history = do.call(rbind, hist[1:iter])))
  }

  list(converged = FALSE, reason = "max_iter reached",
       iter = max_iter, root = x1, f_root = f1,
       history = do.call(rbind, hist))
}

out <- secant_trace(g_prime, x0 = 0, x1 = 2, max_iter = 60)
cat("Algorithm converged: ", out$converged)
cat("Reason: ", out$reason)
cat("Number of iterations:", out$iter)
head(out$history, 10)
tail(out$history, 10)
# 2.1
f <- function(a) {
  x <- c(-1, -a, a, 1)

```

```

X <- cbind(
  1,
  x,
  x^2,
  x^3
)

det(t(X) %*% X)
}
# 2.2
f <- function(a) {
  x <- c(-1, -a, a, 1)
  X <- cbind(1, x, x^2, x^3)
  det(t(X) %*% X)
}

aa <- seq(0, 1, length.out = 2001)
ff <- sapply(aa, f)

plot(aa, ff, type = "l", lwd = 2,
      xlab = "a", ylab = "f(a) = det(t(X) %*% X)",
      main = "Determinant of information matrix vs a")

abline(v = c(0, 1), lty = 2, col = "grey50")
points(c(0, 1), c(f(0), f(1)), pch = 19, col = "red")

f0 <- f(0); f1 <- f(1)
# 2.3
opt <- optimize(f, interval = c(0, 1), maximum = TRUE)

```